Student Name: _____ Roll No.: _____

Branch / Semester: _____ Date: _____

Note: The slab rates, thresholds, and policies used below are illustrative assumptions for this assignment. They are not official tariffs or institutional rules.

Contents

1. Progressive Electricity Billing

2. Hospital Billing and Patient Classification

3. Employee Payroll and Deduction Analysis

4. Retail Billing with Discount

5. Manufacturing Cost and Profitability

6. Examination Result and Grace-Mark Validation

7. Attendance Recovery Computation

8. ATM Transaction Sequence Simulation

9. Warehouse Inventory Reconciliation

10. Student Marks Matrix Analysis

Problem 1: Progressive Electricity Billing

1. Problem Statement

Develop a Python program to calculate a household electricity bill using consumption slabs and a fixed charge.

2. Objectives

Accept units consumed as input.

Calculate charges using progressive slabs.

Add a fixed charge and display the final bill.

3. Assumptions

Units consumed cannot be negative.

First 100 units cost ₹2 per unit; next 100 units cost ₹3 per unit; units above 200 cost ₹5 per unit.

A fixed charge of ₹50 is added.

4. Algorithm

1. Start.
2. Read units consumed.
3. If units are at most 100, charge all units at ₹2.
4. If units are 101–200, charge the first 100 at ₹2 and the rest at ₹3.
5. If units exceed 200, charge the first 100 at ₹2, next 100 at ₹3, and remaining units at ₹5.
6. Add ₹50 fixed charge.
7. Display the bill and stop.

5. Flowchart

```
[START]
  |
[INPUT VALUES]
  |
[VALID INPUT?] --No--> [ERROR MESSAGE] --> [STOP]
  | Yes
[CALCULATE CHARGES]
  |
[APPLY RULE / CHECK]
  |
[DISPLAY BILL]
  |
 [STOP]
```

Flowchart key: square-bracketed labels represent process/input-output steps; question labels represent decisions.

6. Pseudocode

```
START\nINPUT units\nIF units < 0: PRINT invalid\nELSE IF units <= 100: energy = units * 2\nELSE IF units <= 200: energy = 100*2 + (units-100)*3\nELSE: energy = 100*2 + 100*3 + (units-200)*5\nbill = energy + 50\nPRINT bill\nSTOP
```

7. Python Source Code

```
units = int(input("Enter units consumed: "))


if units < 0:
```

```
    print("Invalid units")
else:
    if units <= 100:
        energy_charge = units * 2
    elif units <= 200:
        energy_charge = 100 * 2 + (units - 100) * 3
    else:
        energy_charge = 100 * 2 + 100 * 3 + (units - 200) * 5

    total_bill = energy_charge + 50
    print("Energy charge: Rs.", energy_charge)
    print("Fixed charge: Rs. 50")
    print("Total bill: Rs.", total_bill)
```

8. Test Cases

Test	Input	Expected Result
1	80	Energy charge ₹160; total bill ₹210
2	150	Energy charge ₹350; total bill ₹400
3	250	Energy charge ₹750; total bill ₹800

9. Output Screenshot

Sample run of the program:



10. Conclusion

The program calculates electricity charges correctly for different consumption slabs and includes the fixed charge.

Problem 2: Hospital Billing and Patient Classification

1. Problem Statement

Prepare a hospital bill from consultation, diagnostic, and treatment charges, and apply a concession when the subtotal is at least ₹5,000.

2. Objectives

Read three service charges.

Calculate subtotal and concession.

Display the final amount and patient category.

3. Assumptions

All charges must be zero or positive.

A 10% concession is applied when the subtotal is ₹5,000 or more.

The patient is classified as General below ₹5,000 and High Billing at or above ₹5,000, based on subtotal.

4. Algorithm

1. Start and read consultation, diagnostic, and treatment charges.
2. Check that no charge is negative.
3. Calculate subtotal by adding all charges.
4. If subtotal is at least ₹5,000, calculate a 10% concession; otherwise concession is zero.

5. Subtract concession from subtotal.

6. Display subtotal, concession, final bill, and classification.

7. Stop.

5. Flowchart

```
[START]
  |
[INPUT VALUES]
  |
[VALID INPUT?] --No--> [ERROR MESSAGE] --> [STOP]
  | Yes
[CALCULATE CHARGES]
  |
[APPLY RULE / CHECK]
  |
[DISPLAY BILL]
  |
 [STOP]
```

Flowchart key: square-bracketed labels represent process/input-output steps; question labels represent decisions.

6. Pseudocode

```
START\nINPUT consultation, diagnostics, treatment\nIF any charge < 0: PRINT invalid\nELSE:\n  subtotal = consultation + diagnostics + treatment\n  IF subtotal >= 5000: discount = subtotal * 0.10\n  ELSE: discount = 0\n  final = subtotal - discount\nPRINT subtotal, discount, final, category\nSTOP
```

7. Python Source Code

```
consultation = float(input("Consultation charge: "))
diagnostics = float(input("Diagnostic charge: "))
treatment = float(input("Treatment charge: "))


if consultation < 0 or diagnostics < 0 or treatment < 0:
    print("Invalid charge")
else:
    subtotal = consultation + diagnostics + treatment
```

```
if subtotal >= 5000:
    concession = subtotal * 0.10
    category = "High Billing"
else:
    concession = 0
    category = "General"

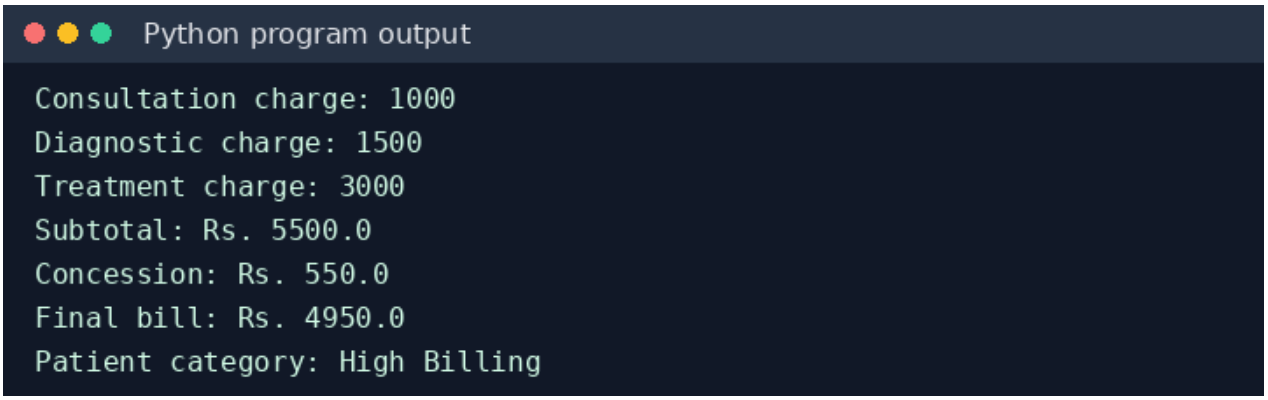
final_bill = subtotal - concession
print("Subtotal: Rs.", round(subtotal, 2))
print("Concession: Rs.", round(concession, 2))
print("Final bill: Rs.", round(final_bill, 2))
print("Patient category:", category)
```

8. Test Cases

Test	Input	Expected Result
1	500, 1000, 2000	Subtotal ₹3500; concession ₹0; final ₹3500; General
2	1000, 1500, 3000	Subtotal ₹5500; concession ₹550; final ₹4950; High Billing

9. Output Screenshot

Sample run of the program:



10. Conclusion

The program adds hospital charges, applies the stated concession rule, and classifies the bill according to the assumed policy.

Problem 3: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate an employee's gross salary from basic pay, allowance, and overtime, then deduct a fixed percentage to find net salary.

2. Objectives

Calculate gross salary.

Apply a deduction rate.

Print a simple salary statement.

3. Assumptions

Basic pay, allowance, and overtime pay are non-negative.

Deduction is 8% of gross salary.

Gross salary equals basic pay plus allowance plus overtime pay.

4. Algorithm

1. Start.

2. Input basic pay, allowance, and overtime pay.

3. Validate that all values are non-negative.

4. Add them to get gross salary.

5. Calculate deduction as 8% of gross salary.

6. Subtract deduction to get net salary.

7. Display the salary statement and stop.

5. Flowchart

```
[START] -> [INPUT DATA] -> [VALIDATE INPUTS]
                                |
                          [PROCESS IN LOOP]
                                |
                          [APPLY CONDITIONS]
                                |
                          [DISPLAY RESULT]
                                |
                             [STOP]
```

Flowchart key: square-bracketed labels represent process/input-output steps; question labels represent decisions.

6. Pseudocode

```
START\nINPUT basic, allowance, overtime\nIF any value < 0: PRINT invalid\nELSE:\n  gross = basic + allowance +
overtime\n  deduction = gross * 0.08\n  net = gross - deduction\n  PRINT gross, deduction, net\nSTOP
```

7. Python Source Code

```
basic = float(input("Basic pay: "))
allowance = float(input("Allowance: "))
overtime = float(input("Overtime pay: "))


if basic < 0 or allowance < 0 or overtime < 0:
    print("Invalid salary details")
else:
    gross = basic + allowance + overtime
    deduction = gross * 0.08
    net = gross - deduction
    print("Gross salary: Rs.", round(gross, 2))
    print("Deduction (8%): Rs.", round(deduction, 2))
    print("Net salary: Rs.", round(net, 2))
```

8. Test Cases

Test	Input	Expected Result
1	15000, 2000, 1000	Gross ₹18000; deduction ₹1440; net ₹16560
2	10000, 0, 0	Gross ₹10000; deduction ₹800; net ₹9200

9. Output Screenshot

Sample run of the program:

Python program output

```
Basic pay: 15000
Allowance: 2000
Overtime pay: 1000
Gross salary: Rs. 18000.0
Deduction (8%): Rs. 1440.0
Net salary: Rs. 16560.0
```

10. Conclusion

The program demonstrates arithmetic operations and conditional validation to prepare a basic salary statement.

Problem 4: Retail Billing with Discount

1. Problem Statement

Calculate a retail bill for multiple items using quantity, unit price, and a threshold-based discount.

2. Objectives

Calculate the total cost of each item.

Add item costs to obtain the subtotal.

Apply a discount when the subtotal reaches the threshold.

3. Assumptions

There are three items.

Quantities and prices cannot be negative.

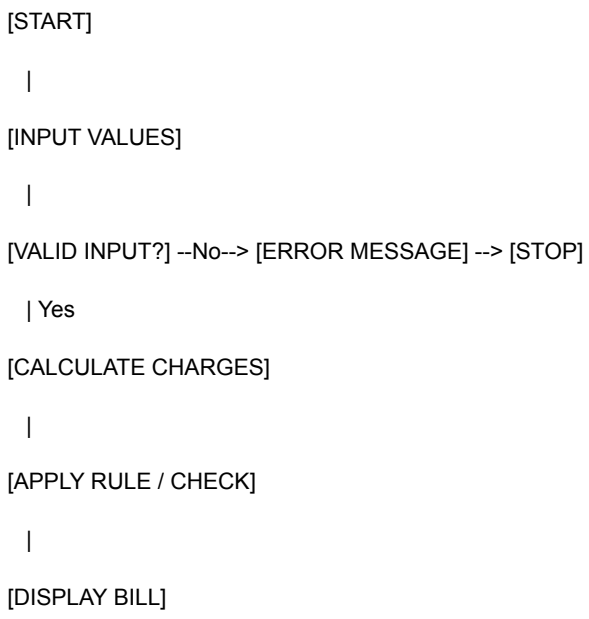
A 5% discount applies when the subtotal is ₹2,000 or more.

No additional taxes are included.

4. Algorithm

1. Start.
2. Repeat for three items: input price and quantity, validate them, and calculate item cost.
3. Add item costs to get subtotal.
4. If subtotal is at least ₹2,000, apply 5% discount; otherwise discount is zero.
5. Calculate final amount and display the bill.
6. Stop.

5. Flowchart



|

[STOP]

Flowchart key: square-bracketed labels represent process/input-output steps; question labels represent decisions.

6. Pseudocode

START\nsubtotal = 0\nREPEAT 3 TIMES:\n INPUT price, quantity\n IF price < 0 OR quantity < 0: PRINT invalid and STOP\n subtotal = subtotal + price * quantity\nIF subtotal >= 2000: discount = subtotal * 0.05\nELSE: discount = 0\nfinal = subtotal - discount\nPRINT subtotal, discount, final\nSTOP

7. Python Source Code

```
subtotal = 0


for item in range(1, 4):
    price = float(input("Enter price of item " + str(item) + ": "))
    quantity = int(input("Enter quantity: "))
    if price < 0 or quantity < 0:
        print("Invalid price or quantity")
        break
    subtotal += price * quantity
else:
    if subtotal >= 2000:
        discount = subtotal * 0.05
    else:
        discount = 0
    print("Subtotal: Rs.", round(subtotal, 2))
    print("Discount: Rs.", round(discount, 2))
    print("Final amount: Rs.", round(subtotal - discount, 2))
```

8. Test Cases

Test	Input	Expected Result
1	500, 2; 300, 1;	Subtotal ₹1500;

	200, 1	discount ₹0; final ₹1500
2	700, 2; 400, 1; 300, 1	Subtotal ₹2100; discount ₹105; final ₹1995

9. Output Screenshot

Sample run of the program:

Python program output

```
Enter price of item 1: 700
Enter quantity: 2
Enter price of item 2: 400
Enter quantity: 1
Enter price of item 3: 300
Enter quantity: 1
Subtotal: Rs. 2100.0
Discount: Rs. 105.0
Final amount: Rs. 1995.0
```

10. Conclusion

The program uses a loop to process three items and a conditional statement to apply a discount.

Problem 5: Manufacturing Cost and Profitability

1. Problem Statement

Calculate total production cost and profit for a batch of products, including material, labour, and machine costs.

2. Objectives

Calculate total production cost.

Calculate sales revenue and profit.

Check whether a target profit is achieved.

3. Assumptions

All costs, quantity, and selling price are non-negative.

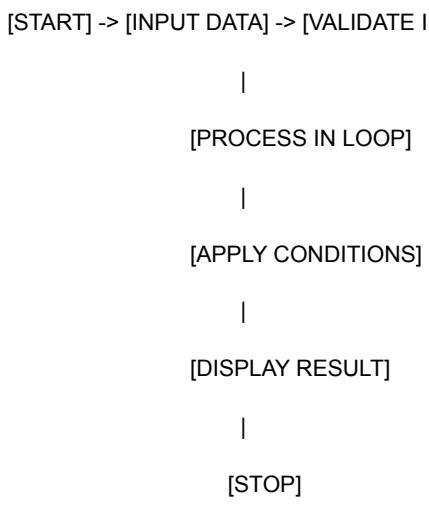
Total cost is the sum of material, labour, and machine costs.

Target profit is ₹2,000.

4. Algorithm

1. Start and read material, labour, machine costs, quantity, and selling price per unit.
2. Validate all values.
3. Calculate total cost and sales revenue.
4. Calculate profit as revenue minus cost.
5. If profit is at least ₹2,000, print target achieved; otherwise print target not achieved.
6. Display the results and stop.

5. Flowchart



Flowchart key: square-bracketed labels represent process/input-output steps; question labels represent decisions.

6. Pseudocode

START\nINPUT material, labour, machine, quantity, selling_price\nIF any value < 0: PRINT invalid\nELSE:\n cost = material + labour + machine\n revenue = quantity * selling_price\n profit = revenue - cost\n IF profit >= 2000: status = target achieved\n ELSE: status = target not achieved\n PRINT cost, revenue, profit, status\nSTOP

7. Python Source Code

```
material = float(input("Material cost: "))
labour = float(input("Labour cost: "))
machine = float(input("Machine cost: "))
quantity = int(input("Quantity produced: "))
selling_price = float(input("Selling price per unit: "))

if min(material, labour, machine, quantity, selling_price) < 0:
    print("Invalid input")
else:
    total_cost = material + labour + machine
    revenue = quantity * selling_price
    profit = revenue - total_cost
    print("Total production cost: Rs.", round(total_cost, 2))
    print("Sales revenue: Rs.", round(revenue, 2))
    print("Profit: Rs.", round(profit, 2))
    if profit >= 2000:
        print("Target profit achieved")
    else:
        print("Target profit not achieved")
```

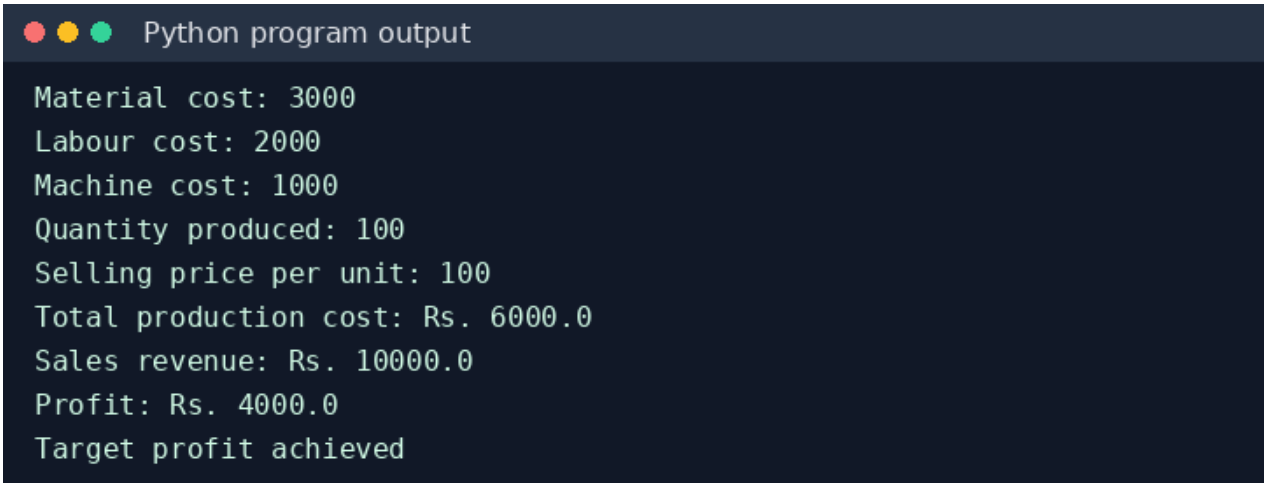
8. Test Cases

Test	Input	Expected Result
1	3000, 2000, 1000, 100, 100	Cost ₹6000; revenue ₹10000; profit ₹4000; target achieved

2	5000, 3000, 2000, 50, 150	Cost ₹10000; revenue ₹7500; loss ₹2500; target not achieved

9. Output Screenshot

Sample run of the program:



10. Conclusion

The program compares revenue with production cost and checks whether the required profit target is met.

Problem 6: Examination Result and Grace-Mark Validation

1. Problem Statement

Calculate a student's total marks and result in three subjects, with a limited grace-mark rule for one-mark shortages.

2. Objectives

Accept marks in three subjects.

Check pass marks in each subject.

Apply a small grace mark only when the defined condition is met.

3. Assumptions

Each subject is out of 100 and pass mark is 35.

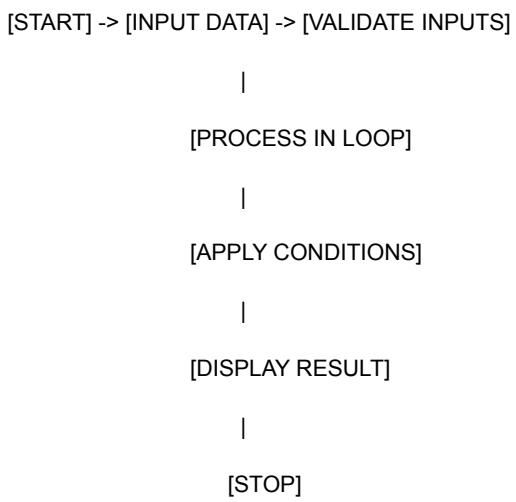
Grace of one mark may be added to exactly one subject scored 34, only if the other two subjects are passed.

Final marks cannot exceed 100.

4. Algorithm

1. Start and input marks for three subjects.
2. Validate that each mark is between 0 and 100.
3. Count subjects with marks below 35.
4. If exactly one subject has 34 and the other two are passed, add one grace mark to that subject.
5. Check all subjects against 35 and display pass/fail and total.
6. Stop.

5. Flowchart



Flowchart key: square-bracketed labels represent process/input-output steps; question labels represent decisions.

6. Pseudocode

START\nINPUT m1, m2, m3\nIF any mark outside 0..100: PRINT invalid\nELSE:\n IF exactly one mark = 34 AND other marks >= 35: add 1 to that mark\n IF all marks >= 35: result = PASS\n ELSE: result = FAIL\n PRINT marks, total, result\nSTOP

7. Python Source Code

```
marks = [int(input("Enter marks for subject " + str(i) + ": "))
         for i in range(1, 4)]


if any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid marks")
else:
    low = [i for i in range(3) if marks[i] < 35]
    if len(low) == 1 and marks[low[0]] == 34:
        other_marks = [marks[i] for i in range(3) if i != low[0]]
        if all(mark >= 35 for mark in other_marks):
            marks[low[0]] = min(100, marks[low[0]] + 1)
            print("One grace mark applied")


    total = sum(marks)
    print("Final marks:", marks)
    print("Total:", total, "/ 300")
    if all(mark >= 35 for mark in marks):
        print("Result: PASS")
    else:
        print("Result: FAIL")
```

8. Test Cases

Test	Input	Expected Result
1	34, 60, 70	Grace applied; final marks [35, 60, 70]; total 165; PASS

2	30, 60, 70	No grace; total 160; FAIL

9. Output Screenshot

Sample run of the program:

Python program output

```
Enter marks for subject 1: 34
Enter marks for subject 2: 60
Enter marks for subject 3: 70
One grace mark applied
Final marks: [35, 60, 70]
Total: 165 / 300
Result: PASS
```

10. Conclusion

The program validates marks and applies grace marks only under the stated conditions before determining the result.

Problem 7: Attendance Recovery Computation

1. Problem Statement

Calculate current attendance percentage and determine the minimum number of consecutive classes a student must attend to reach a target percentage.

2. Objectives

Calculate attendance percentage.

Determine the number of future classes needed.

Handle students who already meet the target.

3. Assumptions

Classes held must be greater than zero.

Classes attended must be between zero and classes held.

Target attendance is 75%. Every future class is attended.

4. Algorithm

1. Start and input classes held and attended.

2. Validate the inputs.

3. Calculate current attendance percentage.

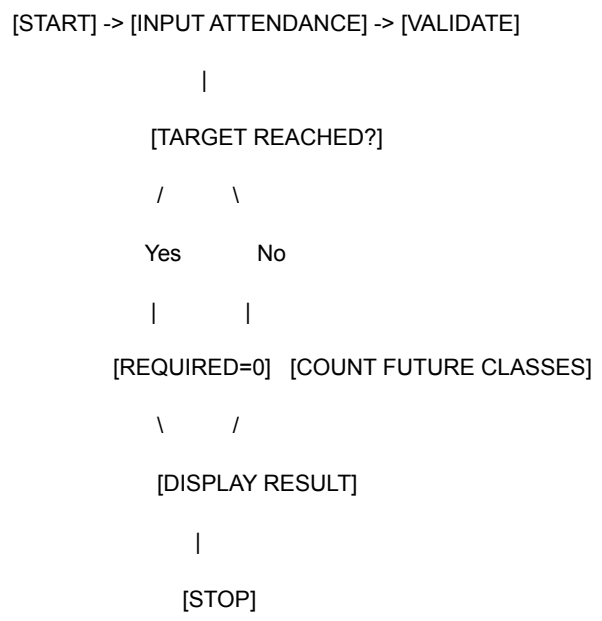
4. If current attendance is at least 75%, required classes are zero.

5. Otherwise increase future attended classes one at a time until attended plus future classes divided by held plus future classes reaches 75%.

6. Display current percentage and required classes.

7. Stop.

5. Flowchart



Flowchart key: square-bracketed labels represent process/input-output steps; question labels represent decisions.

6. Pseudocode

START\nINPUT held, attended\nIF held <= 0 OR attended < 0 OR attended > held: PRINT invalid\nELSE:\n percent = attended / held * 100\n future = 0\n WHILE (attended + future)/(held + future) < 0.75: future = future + 1\n PRINT percent, future\nSTOP

7. Python Source Code

```
held = int(input("Total classes held: "))
attended = int(input("Classes attended: "))
```

```
if held <= 0 or attended < 0 or attended > held:
    print("Invalid attendance details")
else:
    percentage = attended / held * 100
    future = 0
    while (attended + future) / (held + future) < 0.75:
        future += 1
    print("Current attendance:", round(percentage, 2), "%")
    print("Additional classes to attend:", future)
```

8. Test Cases

Test	Input	Expected Result
1	40, 25	Current attendance 62.5%; additional classes 20
2	40, 32	Current attendance 80%; additional classes 0

9. Output Screenshot

Sample run of the program:

```
Python program output
Total classes held: 40
Classes attended: 25
Current attendance: 62.5 %
Additional classes to attend: 20
```

10. Conclusion

The while loop finds the smallest number of future classes needed to reach the target attendance percentage.

Problem 8: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a sequence of deposits and withdrawals, reject invalid transactions, and display the final balance.

2. Objectives

Maintain an account balance.

Process deposit and withdrawal operations.

Reject withdrawals greater than the available balance.

3. Assumptions

Opening balance is ₹1,000.

A transaction amount must be positive.

A withdrawal is accepted only when the amount does not exceed the balance.

4. Algorithm

1. Start with balance ₹1,000.
2. Input the number of transactions.
3. For each transaction, input type and amount.
4. For a positive deposit, add the amount; for a valid withdrawal, subtract it; otherwise reject it.
5. Display the balance after each accepted transaction and at the end.
6. Stop.

5. Flowchart

[START] -> [INPUT DATA] -> [VALIDATE INPUTS]

|

[PROCESS IN LOOP]

|

[APPLY CONDITIONS]

|

[DISPLAY RESULT]

|

[STOP]

Flowchart key: square-bracketed labels represent process/input-output steps; question labels represent decisions.

6. Pseudocode

START\nbalance = 1000\nINPUT number_of_transactions\nREPEAT number_of_transactions TIMES:\n INPUT type, amount\n IF amount <= 0: PRINT invalid\n ELSE IF type is deposit: balance = balance + amount\n ELSE IF type is withdraw AND amount <= balance: balance = balance - amount\n ELSE: PRINT rejected\nPRINT final balance\nSTOP

7. Python Source Code

```
balance = 1000.0
count = int(input("Number of transactions: "))

for i in range(count):
    kind = input("Transaction type (D/W): ").upper()
    amount = float(input("Amount: "))

    if amount <= 0:
        print("Rejected: amount must be positive")
    elif kind == "D":
        balance += amount
        print("Deposit successful")
    elif kind == "W" and amount <= balance:
        balance -= amount
        print("Withdrawal successful")
```

```
    elif kind == "W":
        print("Rejected: insufficient balance")
    else:
        print("Rejected: invalid transaction type")
    print("Current balance: Rs.", round(balance, 2))

print("Final balance: Rs.", round(balance, 2))
```

8. Test Cases

Test	Input	Expected Result
1	D 500; W 200	Final balance ₹1300
2	W 1500	Withdrawal rejected; final balance ₹1000

9. Output Screenshot

Sample run of the program:

Python program output

```
Number of transactions: 2
Transaction type (D/W): D
Amount: 500
Deposit successful
Current balance: Rs. 1500.0
Transaction type (D/W): W
Amount: 200
Withdrawal successful
Current balance: Rs. 1300.0
Final balance: Rs. 1300.0
```

10. Conclusion

The program uses a loop and conditional statements to update a simulated balance and reject invalid transactions.

Problem 9: Warehouse Inventory Reconciliation

1. Problem Statement

Compare opening stock, received stock, and dispatched stock for several products, then calculate expected closing stock and identify shortages.

2. Objectives

Store product stock data in a list.

Calculate expected closing stock.

Compare recorded closing stock with expected stock.

3. Assumptions

Three products are checked.

Opening, received, dispatched, and recorded closing quantities are non-negative integers.

Expected closing stock equals opening stock plus received stock minus dispatched stock.

4. Algorithm

1. Start and create an empty discrepancy list.

2. For each of three products, input opening, received, dispatched, and recorded closing quantities.

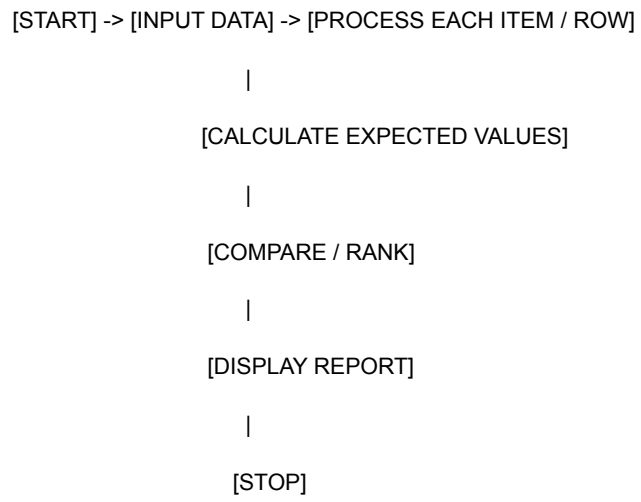
3. Calculate expected closing stock.

4. If dispatched quantity exceeds available stock, report invalid stock movement.

5. Otherwise compare expected and recorded closing stock.

6. Display any discrepancy and stop.

5. Flowchart



Flowchart key: square-bracketed labels represent process/input-output steps; question labels represent decisions.

6. Pseudocode

START\nFOR each of 3 products:\n INPUT opening, received, dispatched, recorded_closing\n available = opening + received\n IF dispatched > available: PRINT invalid movement\n ELSE:\n   expected = available - dispatched\n   difference = recorded_closing - expected\n   IF difference = 0: PRINT matched\n   ELSE: PRINT discrepancy\nSTOP

7. Python Source Code

```
for product in range(1, 4):
    print("Product", product)
    opening = int(input("Opening stock: "))
    received = int(input("Received quantity: "))
    dispatched = int(input("Dispatched quantity: "))
    recorded = int(input("Recorded closing stock: "))

    if min(opening, received, dispatched, recorded) < 0:
        print("Invalid quantity")
    elif dispatched > opening + received:
        print("Invalid movement: dispatch exceeds available stock")
```

```
else:
    expected = opening + received - dispatched
    difference = recorded - expected
    print("Expected closing stock:", expected)
    if difference == 0:
        print("Stock matched")
    else:
        print("Discrepancy:", difference, "(recorded - expected)")
```

8. Test Cases

Test	Input	Expected Result
1	Opening 100, received 20, dispatched 30, recorded 90	Expected 90; stock matched
2	Opening 50, received 10, dispatched 20, recorded 35	Expected 40; discrepancy -5

9. Output Screenshot

Sample run of the program:

Python program output

```
Product 1
Opening stock: 100
Received quantity: 20
Dispatched quantity: 30
Recorded closing stock: 90
Expected closing stock: 90
Stock matched
```

10. Conclusion

The program uses arithmetic and conditional checks to reconcile inventory and flag differences in recorded stock.

Problem 10: Student Marks Matrix Analysis

1. Problem Statement

Analyse a matrix of marks for three students across three subjects and calculate each student's total and average.

2. Objectives

Represent marks using a nested list.

Calculate row-wise totals and averages.

Identify the student with the highest total.

3. Assumptions

There are three students and three subjects.

Each mark is an integer from 0 to 100.

Students are identified as Student 1, Student 2, and Student 3.

4. Algorithm

1. Start and create a 3-by-3 marks matrix.

2. Input marks for each student using nested loops.

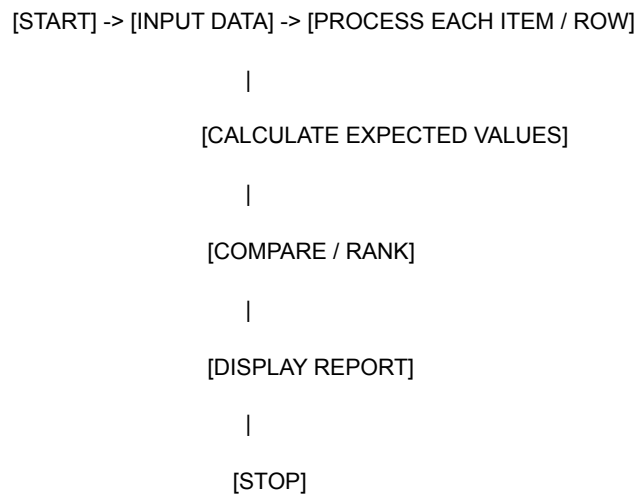
3. Validate each mark.

4. Calculate total and average for each student.

5. Find the highest total and display the student number.

6. Stop.

5. Flowchart



Flowchart key: square-bracketed labels represent process/input-output steps; question labels represent decisions.

6. Pseudocode

START\nCREATE empty matrix\nFOR each of 3 students:\n INPUT 3 subject marks and append as a row\nFOR each row in matrix:\n total = sum(row)\n average = total / 3\n PRINT total and average\nFIND row with maximum sum\nPRINT top student\nSTOP

7. Python Source Code

```
marks = []


for student in range(1, 4):
    row = []
    print("Enter marks for Student", student)
    for subject in range(1, 4):
        mark = int(input("Subject " + str(subject) + ": "))
        row.append(mark)
    marks.append(row)


if any(mark < 0 or mark > 100 for row in marks for mark in row):
    print("Invalid marks: each mark must be from 0 to 100")
else:
```

```
totals = []
for i, row in enumerate(marks, start=1):
    total = sum(row)
    totals.append(total)
    print("Student", i, "total:", total, "average:", round(total / 3, 2))

top_index = totals.index(max(totals))
print("Highest scorer: Student", top_index + 1)
```

8. Test Cases

Test	Input	Expected Result
1	Rows [80,70,90], [60,75,65], [88,91,85]	Totals 240, 200, 264; averages 80, 66.67, 88.0; highest scorer Student 3

9. Output Screenshot

Sample run of the program:

Python program output

```
Enter marks for Student 1
Subject 1: 80
Subject 2: 70
Subject 3: 90
Enter marks for Student 2
Subject 1: 60
Subject 2: 75
Subject 3: 65
Enter marks for Student 3
Subject 1: 88
Subject 2: 91
Subject 3: 85
Student 1 total: 240 average: 80.0
Student 2 total: 200 average: 66.67
Student 3 total: 264 average: 88.0
Highest scorer: Student 3
```

10. Conclusion

The program demonstrates nested lists and loops to calculate student statistics and identify the highest scorer.

Final Conclusion

These ten case studies demonstrate the use of Python variables, data types, operators, conditional statements, loops, strings, lists, nested lists, and basic validation. Each solution follows a problem-solving sequence from understanding the problem to testing the program and reviewing its output. The assumptions are included to make each problem clear and testable.